🗡️ Lonca Görev Panosu API

Fantastik bir maceracılar loncasının görevlerini yöneten REST API. Görevler oluşturulur, maceracılara atanır, önceliğe göre sınıflanır ve durumları takip edilir.

Teknolojiler: Node.js, Express.js, JSON dosya tabanlı veri

Doküman	İçerik
📄 Proje Tanıtım Dokümanı	Amaç, senaryo, mimari
🗂️ Veri Modeli Tasarımı	Görev varlığı, alanlar, kurallar, durum akışı, SQL karşılığı
📘 API Tasarım Dokümanı	Tüm uç noktalar, istek/yanıt örnekleri, doğrulama kuralları, hata kodları
🧪 Test Dokümantasyonu	Otomatik testler ve Postman test senaryoları
🧭 Kullanım Kılavuzu	Adım adım kurulum, sık yapılan işlemler, sorun giderme

Teslim formu dosyaları (`docs/teslim/`): proje tanıtım dokümanı (2 sayfa) ve Postman test ekran görüntüleri, PDF ve Word olarak.

Kurulum

```
git clone https://github.com/dckaratas/lonca-api.git
cd lonca-api
npm install
npm start          # veya geliştirme için: npm run dev
npm test           # otomatik testler
```

Sunucu varsayılan olarak `http://localhost:3000` adresinde çalışır.

Ortam değişkeni	Varsayılan	Açıklama
`PORT`	`3000`	Sunucu portu
`CORS_ORIGIN`	`*`	API'ye erişebilecek web adresi. API internete açılacaksa web panelinin adresiyle kısıtlanmalıdır (ör. `https://lonca-web.netlify.app`).

> **Not:** API'de kimlik doğrulama yoktur; eğitim projesi olarak yerel kullanım içindir.

Klasör Yapısı

```
src/
├── server.js                    # Sunucuyu başlatır
├── app.js                       # Express uygulaması, middleware'ler, hata yakalayıcı
├── constants.js                 # Öncelik ve durum değerleri
├── routes/
│   ├── taskRoutes.js            # /tasks routing
│   └── reportRoutes.js          # /reports routing
├── controllers/
│   ├── taskController.js        # CRUD, filtreleme, arama, sayfalama
│   └── reportController.js      # Raporlar
├── middleware/
│   ├── logger.js                # İstek loglama middleware'i
│   └── validateTask.js          # Doğrulama middleware'leri
└── data/
    ├── tasks.json               # Veri dosyası
    └── store.js                 # JSON okuma/yazma
test/api.test.js                 # Otomatik testler (node:test)
logs/requests.log                # Logger çıktısı (otomatik oluşur)
```

Veri Modeli — Görev (Task)

Alan	Tip	Açıklama
id	number	Otomatik artan
title	string	**Zorunlu** — görev başlığı
description	string	Görev açıklaması
priority	`critical` | `high` | `medium` | `low`	Öncelik (varsayılan `medium`). Arayüzde lonca rütbesi olarak gösterilir: S / A / B / C
status	`open` | `in_progress` | `completed`	Görev durumu
assignee	string | null	Görevi üstlenen maceracı
reward	number	Altın ödülü
createdAt / updatedAt	ISO tarih	Zaman damgaları

Endpoint'ler

Method	Endpoint	Açıklama
GET	`/tasks`	Tüm görevleri listele
GET	`/tasks/:id`	Görev detayı
POST	`/tasks`	Yeni görev ekle
PUT	`/tasks/:id`	Görevi güncelle
DELETE	`/tasks/:id`	Görevi sil
GET	`/tasks/search?keyword=`	Başlık ve açıklamada arama

Method	Endpoint	Açıklama
GET	`/tasks/assignee/:name`	Bir maceracının görevleri
GET	`/reports/completed`	Tamamlanan görevler raporu
GET	`/reports/pending`	Bekleyen görevler raporu
GET	`/reports/summary`	Genel özet raporu

Filtreleme, sıralama ve sayfalama

`GET /tasks` (ve arama/atanan kişi uç noktaları) şu sorgu parametrelerini kabul eder:

Parametre	Örnek	Açıklama
`status`	`?status=completed`	Duruma göre filtre
`priority`	`?priority=high`	Önceliğe göre filtre
`sort`, `order`	`?sort=createdAt`, `?sort=reward&order=desc`	Sıralama (`createdAt`, `reward`, `priority`, `title`)
`page`, `limit`	`?page=1&limit=10`	Sayfalama; yanıt `{ data, pagination }` biçiminde döner

Ayrıntılar ve yanıt örnekleri: API Dokümantasyonu.

Örnek istek — Görev ekleme

```
POST /tasks
Content-Type: application/json

{
  "title": "Kuzey Mağarasındaki Goblinleri Temizle",
  "description": "Goblin sürüsünü dağıt.",
  "priority": "low",
  "assignee": "Aria Gümüşok",
  "reward": 50
}
```

Doğrulama Middleware'i

`src/middleware/validateTask.js` gelen veriyi controller'dan önce kontrol eder ve hatalı istekte **400** döner. `title` zorunlu ve en fazla 100 karakter; `description` metin; `priority` ve `status` yalnızca tanımlı değerler; `assignee` boş olmayan metin veya `null`; `reward` 0 veya daha büyük sayı olmalıdır. Sorgu parametreleri ve `:id` de doğrulanır. Birden fazla hata varsa hepsi `details` dizisinde listelenir:

```
{
  "error": "title alanı zorunludur",
  "details": ["title alanı zorunludur", "priority şunlardan biri olmalıdır: critical, high, medium, low"]
}
```

Logger Middleware

Her istek için method, endpoint, zaman damgası, durum kodu ve süre hem konsola hem `logs/requests.log` dosyasına yazılır:

```
[2026-10-10T12:04:29.948Z] PUT /tasks/6 200 - 1ms
```

Testler

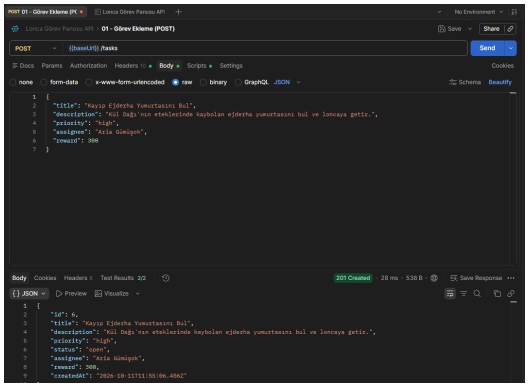
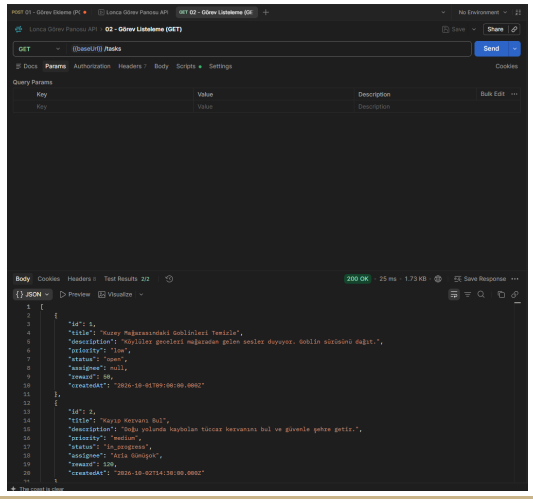
Otomatik testler

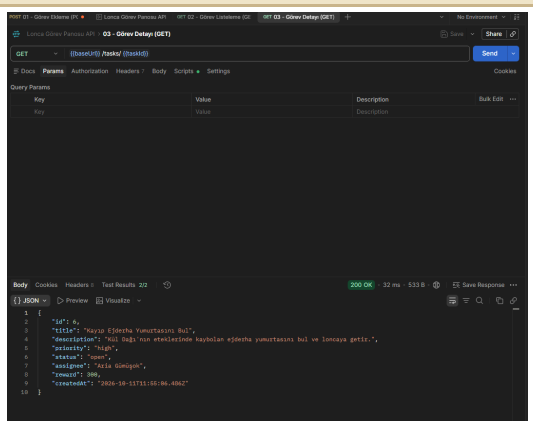
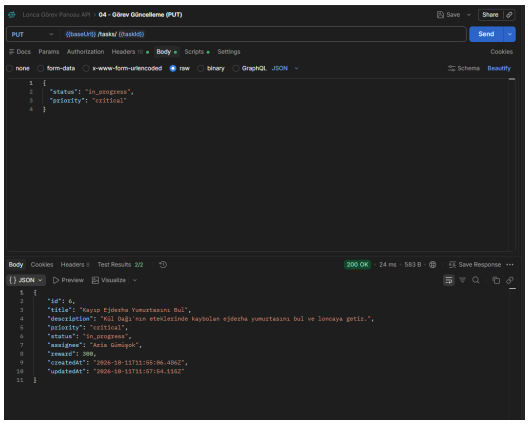
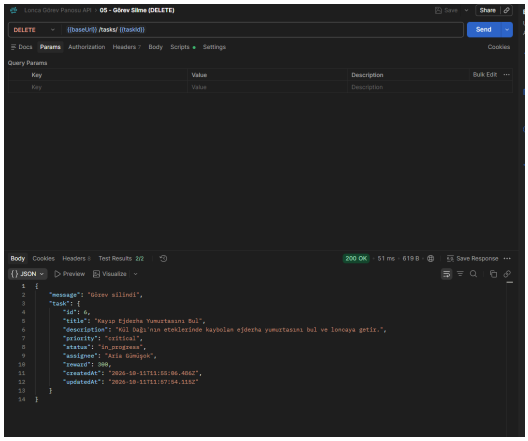
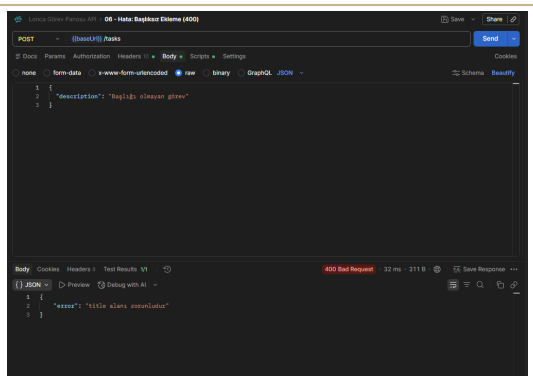
```
npm test
```

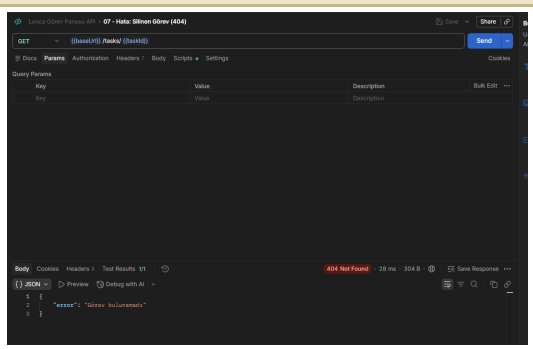
11 test; CRUD akışını, doğrulama kurallarını, filtreleme/arama/sayfalamayı ve raporları kontrol eder. Testler verinin geçici bir kopyasında çalışır, `tasks.json` değişmez. Ayrıntılar: Test Dokümantasyonu.

Postman

Tüm istekler `docs/lonca-api.postman_collection.json` koleksiyonundadır (Postman → **Import**). İstekler sırayla çalıştırılmalıdır: ekleme isteği oluşan görevin id'sini `{{taskId}}` değişkenine kaydeder, sonraki istekler bu görevi kullanır. Her isteğin durum kodunu doğrulayan testleri vardır.

#	İstek	Beklenen	Ekran görüntüsü
1	`POST /tasks` — Görev ekleme	201 Created	
2	`GET /tasks` — Görev listeleme	200 OK	

#	İstek	Beklenen	Ekran görüntüsü
3	`GET /tasks/:id` — Görev detayı	200 OK	
4	`PUT /tasks/:id` — Görev güncelleme	200 OK	
5	`DELETE /tasks/:id` — Görev silme	200 OK	
6	`POST /tasks` — Başlıksız ekleme	400 Bad Request	

#	İstek	Beklenen	Ekran görüntüsü
7	`GET /tasks/:id` — Silinmiş görev	404 Not Found	

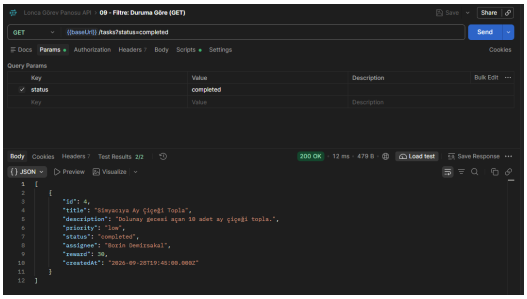
Logger middleware'in bu isteklerdeki konsol çıktısı:

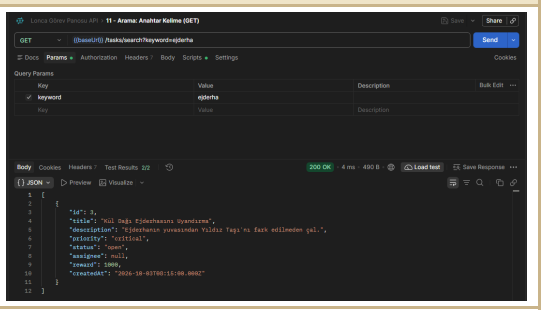
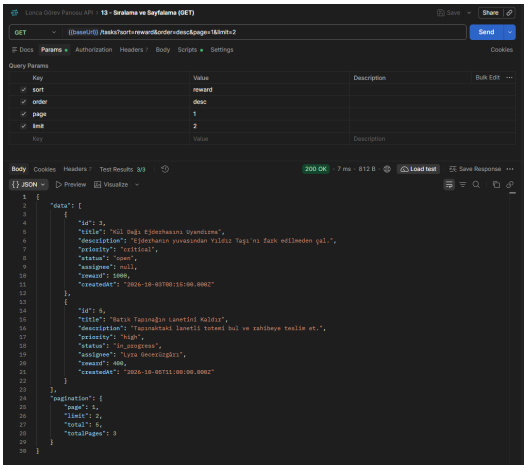
```
PS C:\Users\dgn10\Desktop\Dev\tnc\lonca-api> npm start

> lonca-api@1.0.0 start
> node src/server.js

Lonca API http://localhost:3000 adresinde çalışıyor
[2026-10-11T11:53:23.431Z] POST /tasks 201 - 18ms
[2026-10-11T11:53:23.587Z] GET /tasks 200 - 4ms
[2026-10-11T11:53:23.651Z] GET /tasks/6 200 - 5ms
[2026-10-11T11:53:23.722Z] PUT /tasks/6 200 - 3ms
[2026-10-11T11:53:23.809Z] DELETE /tasks/6 200 - 3ms
[2026-10-11T11:53:23.884Z] POST /tasks 400 - 1ms
[2026-10-11T11:53:23.964Z] GET /tasks/6 404 - 3ms
[2026-10-11T11:55:06.487Z] POST /tasks 201 - 2ms
[2026-10-11T11:56:24.774Z] GET /tasks 200 - 3ms
[2026-10-11T11:57:14.677Z] GET /tasks/6 200 - 3ms
[2026-10-11T11:57:54.116Z] PUT /tasks/6 200 - 2ms
[2026-10-11T11:58:38.195Z] DELETE /tasks/6 200 - 3ms
[2026-10-11T11:59:17.777Z] POST /tasks 400 - 0ms
[2026-10-11T11:59:43.383Z] GET /tasks/6 404 - 2ms
```

Filtreleme, arama, sayfalama ve raporlama isteklerinden örnekler (koleksiyondaki diğer istekler 10, 12, 14, 15, 17 ve 18'dir):

#	İstek	Beklenen	Ekran görüntüsü
9	`GET /tasks?status=completed` — Duruma göre filtre	200 OK	

#	İstek	Beklenen	Ekran görüntüsü
11	`GET /tasks/search?keyword=ejderha` — Arama	200 OK	
13	`GET /tasks?sort=reward&order=desc&page=1&limit=2` — Sıralama ve sayfalama	200 OK	
16	`GET /reports/summary` — Genel özet raporu	200 OK	